

Query optimization basics

COMPLETE LEARNING GUIDE

What You'll Learn

A query that runs in milliseconds on a development database with a thousand rows can take minutes on a production database with tens of millions — and the gap between those two numbers is almost always explained by whether the database can use an index, and whether the application is issuing far more queries than it needs to. This guide covers how a relational database actually executes a query (parsing, planning, executing), how to read an EXPLAIN ANALYZE plan to find out why a query is slow instead of guessing, how B-tree indexes work and when they help or hurt, and the single most common performance bug in ORM-based applications: the N+1 query problem. The goal is to build the habit of measuring before optimizing, rather than adding indexes or rewriting queries speculatively.

1. How a Query Actually Executes

When a SQL query arrives, the database doesn't run it literally as written — it parses the SQL into an internal representation, then a query planner/optimizer considers multiple possible execution strategies (which index to use, if any; which order to join tables in; whether to sort before or after filtering) and picks the one it estimates will be cheapest, based on statistics it keeps about the table's size and data distribution. Only then does the executor actually run that chosen plan against the data. This means the exact same SQL query can execute completely differently — fast or slow — depending on the table's size, its indexes, and how recently the database's internal statistics were updated.

2. Reading EXPLAIN ANALYZE

EXPLAIN shows the query plan the optimizer chose without running the query; EXPLAIN ANALYZE actually executes it and reports real timing and row counts alongside the plan, which is what you want when diagnosing a genuinely slow query. The most important thing to look for is a 'Seq Scan' (sequential scan — reading every row in the table) on a large table where you expected an 'Index Scan' — that mismatch is the single most common cause of an unexpectedly slow query, and usually means either the needed index doesn't exist, or the query is written in a way that prevents the existing index from being used.

SQL

```
EXPLAIN ANALYZE
SELECT o.order_id, c.full_name, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'pending'
ORDER BY o.placed_at DESC
LIMIT 20;

-- Look for lines like:
--   Seq Scan on orders  (cost=0.00..18334.00 rows=520000)  <- red flag on a big
tableIndex Scan using idx_orders_status on orders (cost=0.42..8.60 rows=12) <-
healthy
```

3. How B-Tree Indexes Work, and Their Real Cost

A standard database index (typically a B-tree) is a separate, sorted data structure that lets the database find rows matching a WHERE condition without scanning the entire table — conceptually similar to a book's index letting you jump straight to a topic instead of reading every page. This lookup speed isn't free: every INSERT, UPDATE, or DELETE on the table must also update every index on it, so a table with ten indexes is measurably slower to write to than one with two. Indexes also consume disk space and memory. The practical rule is to index columns that are frequently filtered, joined, or sorted on, and to avoid indexing columns that are rarely queried or that change on almost every write.

- Index columns used in WHERE, JOIN ON, and ORDER BY clauses frequently.
- Every index speeds up reads but slows down writes (INSERT/UPDATE/DELETE) and uses disk space.
- Foreign key columns should almost always be indexed — they're used in joins constantly.
- A low-cardinality column (e.g. a boolean with only two values) usually benefits less from an index than a high-cardinality one.

4. The N+1 Query Problem

The N+1 problem is the most common performance bug in applications using an ORM, and it's subtle because it's invisible in the code — it only shows up when you look at the actual queries being sent to the database. It happens when code fetches a list of N parent records with one query, then loops over them and fetches each one's related data with a separate query — producing 1 + N total queries instead of 2 (or 1, with a join). At a small N this is unnoticeable; at a realistic N (a page listing 50 orders, each triggering a separate query for its customer) it can add hundreds of milliseconds or more of pure network round-trip latency to a single page load.

- Symptom: a page or endpoint issues one query per item in a list, instead of one query total.

- Fix: eager-load related data with a JOIN or the ORM's eager-loading feature (e.g. Sequelize's ``include``, Prisma's ``include``) up front.
- Detecting it requires actually looking at the query log or count for a request, not just reading the ORM code — the bug is invisible in the source.

5. Avoid Over-Fetching: `SELECT *` and Missing `LIMIT`

`SELECT *` pulls every column, including large ones (a text blob, a JSON column) the application may not even use for a given request, wasting network bandwidth and memory on both ends. Explicitly listing only the needed columns is both more efficient and self-documenting about what a query actually depends on. Similarly, any query that could return an unbounded number of rows should carry an explicit `LIMIT` (with pagination), since a query that works fine at 200 rows in development can become a multi-second, memory-heavy query at 2 million rows in production.

6. Composite Indexes and Column Order

A composite index spans multiple columns, and — critically — the order of columns in the index definition matters. An index on `(status, placed_at)` can efficiently serve a query filtering only on `status`, or filtering on `status` and sorting/filtering by `placed_at`, but it generally cannot efficiently serve a query that filters only on `placed_at`, because a B-tree index is only sorted usefully by its leading column first. This means composite index column order should be chosen based on the most common query patterns actually run against that table, not alphabetically or arbitrarily.

7. A Practical Optimization Checklist

When a query is slow, the productive order of investigation is: run `EXPLAIN ANALYZE` and look for sequential scans on large tables; check whether a function is being applied to an indexed column in the `WHERE` clause (`WHERE LOWER(email) = ...` prevents a plain index on `email` from being used, unless a functional index on `LOWER(email)` exists); check whether the application is issuing `N+1` queries by looking at the actual query log for one request; and only after confirming the specific cause, add or adjust an index — adding indexes speculatively without measuring first often makes writes slower without meaningfully helping the read that was actually slow.

Common Pitfalls

- Fetching a list and then looping over it to fetch each item's related data separately (the `N+1` problem) instead of eager-loading with a join.
- Adding an index to every column 'just in case,' which slows down every write without necessarily speeding up the reads that matter.
- Applying a function to an indexed column in a `WHERE` clause (`WHERE LOWER(email) = 'x'`) without a matching functional index, silently forcing a full table scan.

- Guessing at performance fixes without ever running EXPLAIN ANALYZE to see what the database is actually doing.
- Using SELECT * on wide tables, pulling large unused columns across the network on every query.
- Running an unbounded query with no LIMIT against a table that will keep growing, working fine in development and failing under production data volume.

Key Takeaways

- Always measure with EXPLAIN ANALYZE before optimizing — a sequential scan on a large table is the most common red flag.
- Indexes speed up reads but slow down writes and use storage — index deliberately based on real query patterns, not every column.
- The N+1 query problem is invisible in ORM code and only shows up in the query log — eager-load related data instead of looping and fetching.
- Composite index column order matters — an index on (a, b) doesn't efficiently serve a query filtering only on b.
- Avoid SELECT * and always paginate/LIMIT queries that could return unbounded result sets.

Learn More

PostgreSQL Documentation — Using EXPLAIN

<https://www.postgresql.org/docs/current/using-explain.html>

Use The Index, Luke! — SQL Indexing Guide

<https://use-the-index-luke.com/>

PostgreSQL Documentation — Index Types

<https://www.postgresql.org/docs/current/indexes-types.html>